ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

BÁO CÁO MILESTONE3

MÔN: KIẾN TRÚC MÁY TÍNH

Design of Pipelined RISC-V Processors

Giảng viên hướng dẫn:

Trần Hoàng Linh - Nguyễn Trung Hiếu

Lớp: L01_L02

Nhóm: 19

Sinh viên thực hiện:

STT	Họ tên	Mã số sinh viên
1	Phan Bảo Huân	2311114
2	Trần Chí Nhân	2312450
3	Nguyễn Trung Hiếu	2310964

Thành phố Hồ Chí Minh – 10/2025

MỤC LỤC

I. Giới thiệu 1

II. Thiết kế 2

1. Thiết kế các thanh ghi pipelined 2

Thanh ghi pipelined IF_ID 2

Thanh ghi pipelined ID_EX 2

Thanh ghi pipelined EX_MEM 2

Thanh ghi pipelined MEM_WB 2

2. Thiết kế khối Control Unit (decoder), Hazard Unit, Branch Unit 2

Phân tích thiết kế khối Control Unit 2

Control Unit Specification 3

Phân tích thiết kế Hazard Unit 3

Hazard Unit Specification 4

Phân tích thiết kế Branch Unit 4

Branch Unit Specification 4

3. Model Non-forwarding 5

Sơ đồ khối của non - forwarding 5

Kết quả mô phỏng trên server 9

4. Model Forwarding 10

Sơ đồ khối của forwarding 10

Kết quả mô phỏng trên server 11

..... 12

..... 12

5. Model Two-bit Dynamic Branch Prediction 13
Sơ đồ khối của thiết kế 13
Kết quả mô phỏng trên server 15

6. Kết quả so sánh với các mô hình đã làm 16
Bảng so sánh hiệu năng của bộ xử lý pipelined 16

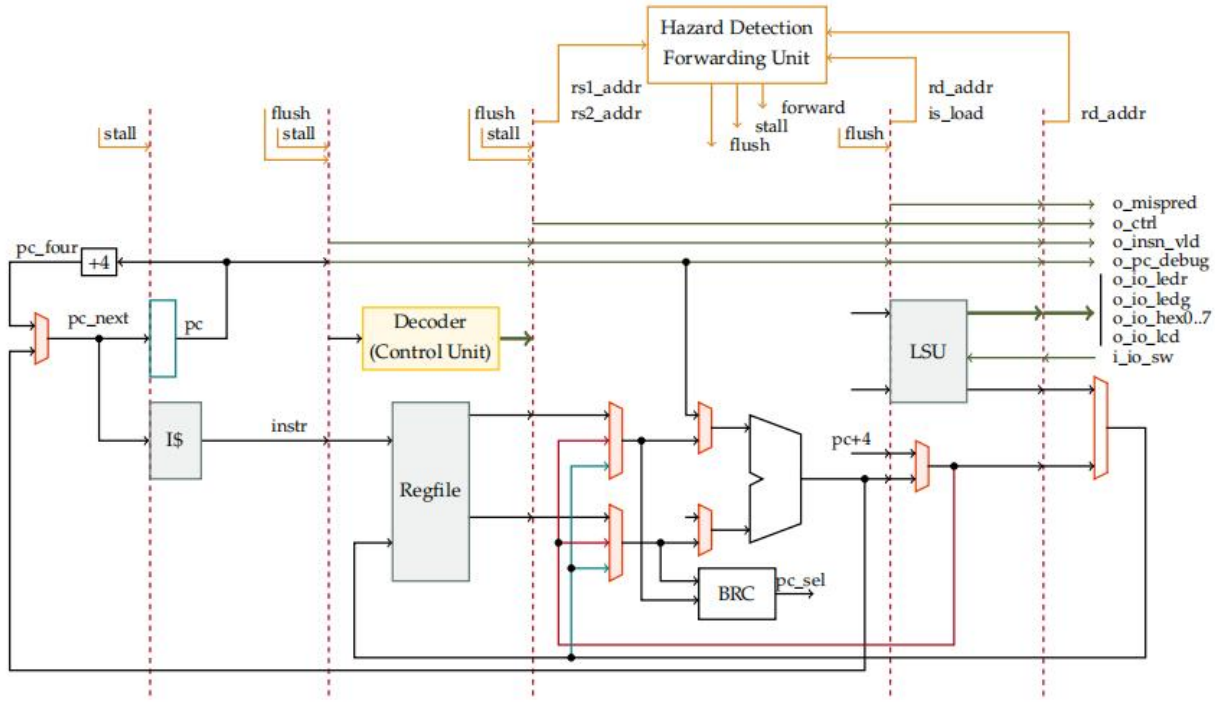
III. Đánh giá 16

DANH MỤC HÌNH ẢNH

Hình 1 : Pipelind Processor 1
Hình 2 : IF Stage 5
Hình 3 : ID Stage 6
Hình 4 : EX Stage 7
Hình 5 : MEM Stage 8
Hình 6 : WB Stage 9
Hình 7 : Sơ đồ khối của non - forwarding 9
Hình 8 : Kết quả chạy trên server 9
Hình 9 : Kết quả đo trên server 10
Hình 10 : Sơ đồ khối forwarding 11
Hình 11 : Kết quả chạy trên server 12
Hình 12 : Kết quả đo trên server 12
Hình 13 : Sơ đồ khối Two-bit Dynamic Branch Prediction 14
Hình 14 : Kết quả chạy trên server 15
Hình 15 : Kết quả chạy trên server 15

I. Giới thiệu

Design of Pipelined RISC-V Processors



Hình 1: Pipelind Processor

Báo cáo thí nghiệm này trình bày về thiết kế của một bộ xử lý pipelined thực hiện ISA RV32I và có thể được tổng hợp trên FPGA. Bộ xử lý cho phép thực hiện song song nhiều lệnh nhằm tăng hiệu năng so với single cycle như ở milestone 2. Ngoài ra bộ xử lý còn tích hợp nhiều khối chức năng để có thể xử lý các loại hazard có thể xảy ra trong quá trình thực hiện các câu lệnh.

II. Thiết kế

- Vì milestone này làm dựa trên những gì đã có sẵn ở milestone 2 trước đó nên nhóm sẽ dùng lại những thiết kế giống và thiết kế thêm những khối chức năng khác có trong milestone 3 này.

1. Thiết kế các thanh ghi pipelined

- Ở tuần đầu tiên nhóm sẽ thiết kế các thanh ghi pipelined giữa các tầng (IF, ID, EX, MEM, WB) vì các khối ALU, BRC, regfile, LSU, memory thiết kế như nhau.

❖ **Thanh ghi pipelined IF_ID**

- Thanh ghi pipelined này nằm giữa pha lấy lệnh và giải mã, có tác dụng lưu trữ và truyền tải các dữ liệu instruction và pc của tầng IF để tầng ID đọc và giải mã dữ liệu, nếu không có thanh ghi này thì dữ liệu ở chu kỳ cũ có thể bị ghi đè lên ở chu kỳ tiếp theo. Ngoài ra còn có thêm tín hiệu flush, stall và insn_vld có chức năng flush, stall và kiểm tra câu lệnh có hợp lệ hay không.

❖ **Thanh ghi pipelined ID_EX**

- Thanh ghi pipelined này nằm giữa pha giải mã và thực thi có tác dụng lưu trữ các tín hiệu điều khiển từ khối decoder (control unit) cho các tầng EX và tầng MEM, ngoài ra nó còn chứa các thông tin đã được giải mã ở tầng trước.

❖ **Thanh ghi pipelined EX_MEM**

- Thanh ghi pipelined này nằm giữa pha thực thi và pha truy cập bộ nhớ, có tác dụng lưu các kết quả tính toán của ALU và dữ liệu của thanh ghi nguồn 2 (rs2) để chuẩn bị cho các lệnh ghi vào bộ nhớ (store). Ngoài ra thanh ghi còn có nhiệm vụ chuyển tiếp các tín hiệu điều khiển cần thiết cho tầng tiếp theo.

❖ **Thanh ghi pipelined MEM_WB**

- Thanh ghi pipelined này nằm giữa pha truy cập bộ nhớ và pha ghi hồi về, có tác dụng lưu dữ liệu vừa đọc từ bộ nhớ (lệnh load) hoặc kết quả tính toán từ ALU, đồng thời có các tín hiệu điều khiển ghi hồi tiếp để cập nhật vào tập thanh ghi.

2. Thiết kế khối Control Unit (decoder), Hazard Unit, Branch Unit

❖ **Phân tích thiết kế khối Control Unit**

- Khối Control Unit ở bài này chỉ còn ở pha ID, nó nhận đầu vào là instruction ở pha IF và trả ra các tín hiệu điều khiển tương ứng cho từng giai đoạn. Dựa vào opcode, function3 và function 7 của lệnh, khối này sẽ giải mã và chọn các tín hiệu như: chọn toán hạng cho ALU, mã lệnh ALU, tín hiệu cho phép ghi cũng như các tín hiệu xử lý rẻ nhánh.

❖ Control Unit Specification

Signal name	Width	Direction	Description
instr_ID	32	input	Instruction 32 bit
br_un_ID	1	output	Compare unsign (1 if unsign)
rd_wren	1	output	Write enable (1 if writing)
opa_sel_ID	1	output	Operand select a (0 if rs1, 1 if PC)
opb_sel_ID	1	output	Operand select b (0 if rs1, 1 if immediate)
alu_op_ID	4	output	Select ALU operation
mem_wren_ID	1	output	Write enable (1 if writing)
type_sel_ID	3	output	Tpye select
wb_sel_ID	1	output	Select data to write back
is_branch_ID	1	output	Branch instruction flag (1 if branch)
is_jal_ID	1	ouput	Jump and Link instruction flag (1 if JAL)
is_jalr_ID	1	output	Jump and Link Register instruction flag (1 if JALR)
o_ctrl_ID	1	output	Control flow instruction
is_ld_ID	1	output	Load instruction flag (1 if Load)

❖ Phân tích thiết kế Hazard Unit

- Khối Hazard unit chịu trách nhiệm phát hiện và giải quyết các xung đột trong pipelined. Đối với Data hazard, nó có nhiệm vụ so sánh địa chỉ thanh ghi nguồn (rs1, rs2) của lệnh hiện tại với địa chỉ thanh ghi đích (rd) của các lệnh trong 3 pha trước đó (rd_EX, rd_MEM, rd_WB). Đối với Control hazard, nó nhận tín hiệu pc_taken để flush các lệnh sai trong thanh ghi IF_ID và ID_EX.

- Khối Hazard unit được cấu tạo từ 6 bộ so sánh 5bit và các cổng logic AND, OR để sinh ra các tín hiệu stall/flush phù hợp.

❖ Hazard Unit Specification

Signal name	Width	Direction	Description
rs1_addr_ID	5	input	Address of the first source register
rs2_addr_ID	5	input	Address of the second source register
rd_x	5	input	Address of the destination register (x: EX, MEM and WB)
rd_wren_x	1	input	Write enable for the destination register (x: EX, MEM, WB) (1 if writing)
pc_taken_EX	1	input	Branch taken signal from EX stage (1 if branch)
stall_PC	1	output	PC stall signal (1 if stall)
stall_IF_ID	1	ouput	IF/ID pipeline register stall (1 if stall)
flush_IF_ID	1	output	IF/ID pipeline register flush (1 if flush)
flush_ID_EX	1	output	ID/EX pipeline register flush (1 if flush)

❖ Phân tích thiết kế Branch Unit

- Khối Branch unit nằm trong pha EX, có nhiệm vụ quyết định xem chương trình có cần thực hiện rẽ nhánh hay tiếp tục thực hiện lệnh tiếp theo. Nó tạo ra tín hiệu pc_sel để lựa chọn giá trị tiếp theo của PC là PC + 4 hoặc là PC của lệnh nhảy. Ngoài ra khối này còn nhận thêm 2 tín hiệu khác là jal, jalr.

❖ Branch Unit Specification

Signal name	Width	Direction	Description
type_sel_EX	3	input	Branch type select
br_less_EX	1	input	Less-than flag (1 if rs1 < rs2)
br_equal_EX	1	input	Equal flag (1 if rs1 = rs2)
is_jal_EX	1	input	Jump and Link instruction flag (1 if JAL)
is_jalr_EX	1	input	Jump and Link Register instruction flag

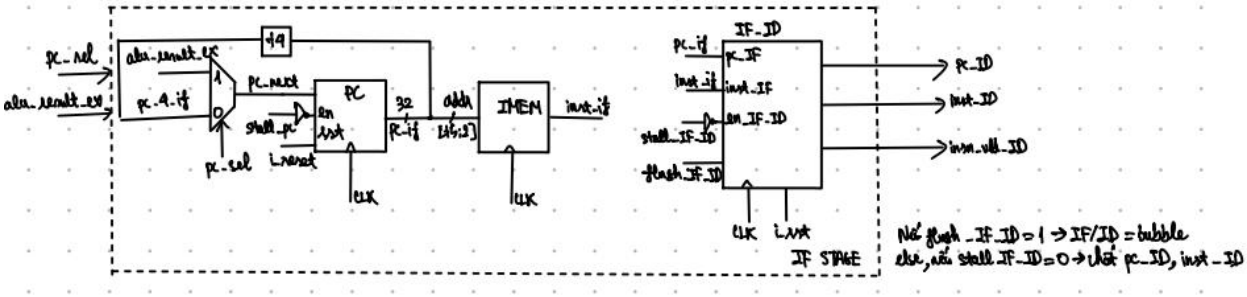
			(1 if JALR)(1 if branch)
is_branch_EX	1	input	Branch enable control (1 if enable)
pc_sel	1	ouput	Pc select (1 if branch, 0 if Pc + 4)

3. Model Non-forwarding

- Trong model này nhóm sẽ sử dụng non-forwarding, khi phát hiện xung đột dữ liệu (data hazard) bộ xử lý sẽ dừng lại (stall) và chèn các lệnh NOP cho đến khi dữ liệu sẵn sàng. Trong model này còn tích hợp thêm khối Hazard unit đã được thiết kế phía trên để phải hiện ra các xung đột trong khi thực hiện lệnh. Khối Hazard unit này sẽ so sánh địa chỉ thanh ghi nguồn (rs1, rs2) của lệnh đang ở pha giải mã ID với địa chỉ thanh ghi đích (rd) của các lênh đang nằm ở các pha sau như EX, MEM, WB. Điều kiện để phát hiện data hazard khi rs1_ID hoặc rs2_ID trùng với thanh ghi đích ở các pha sau (rd_EX, rd_MEM, rd_WB), ngoài ra còn phải kiểm tra các chân cho phép ghi để xem lệnh trước đó có ảnh hưởng tới thanh ghi đích hay không. Còn control hazard thì đối với các lệnh điều khiển (branch, jump), nhóm sử dụng dự đoán không nhảy, bộ xử lý giả sử điều kiện rẽ nhánh là sai và địa chỉ tiếp theo là PC + 4. Tại pha EX, khối BRC và ALU sẽ kiểm tra xem dự đoán là đúng hay sai, nếu dự đoán sai thì tín hiệu pc_taken_EX sẽ được bật lên mức 1. Khối regfile còn được tích hợp thêm 2 con mux nhằm mục đích lấy thẳng giá trị rd vừa được tính toán ở câu lệnh trước, giảm thêm số chu kỳ phải chờ.

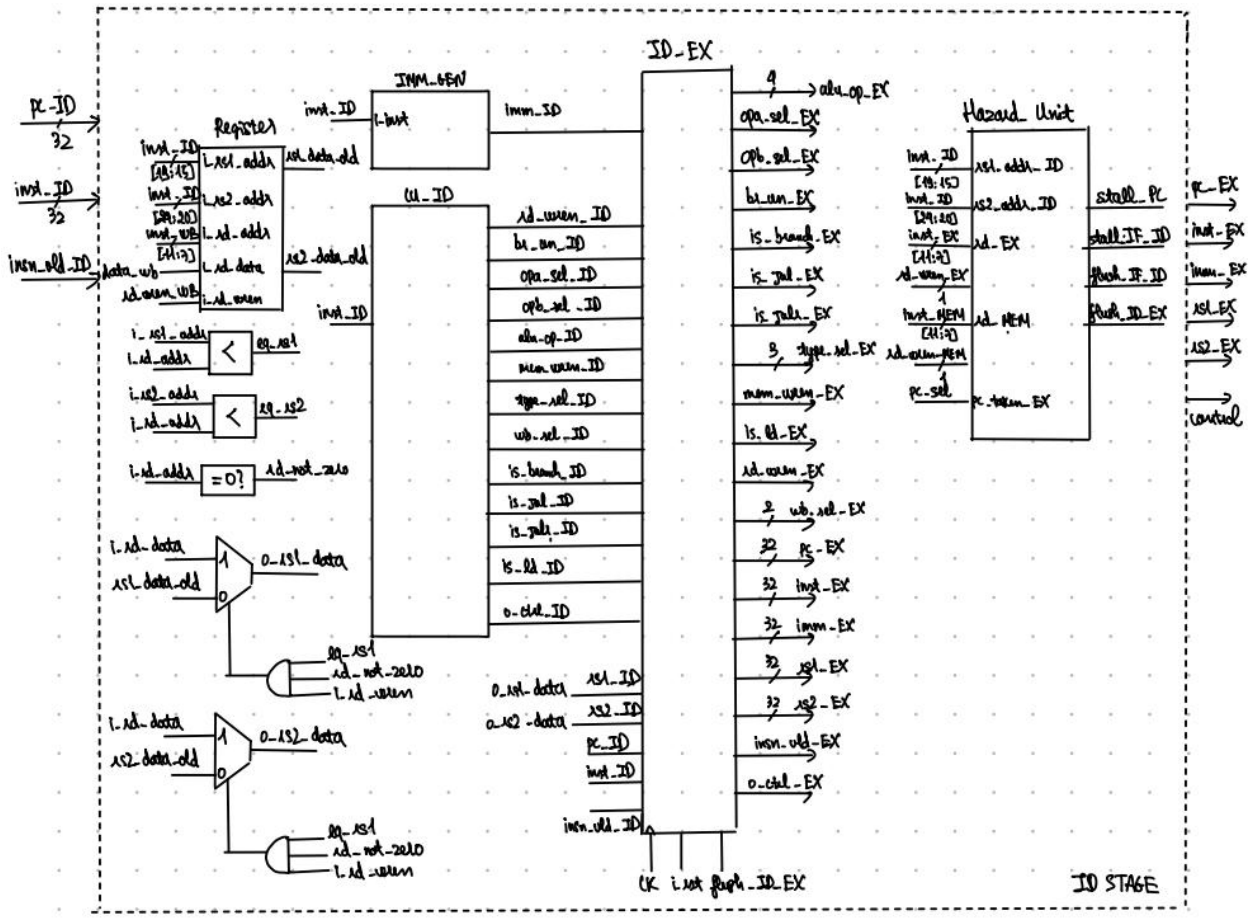
❖ Sơ đồ khối của non - forwarding

- <u>IF Stage:</u>



Hình 2: IF Stage

- <u>ID Stage:</u>



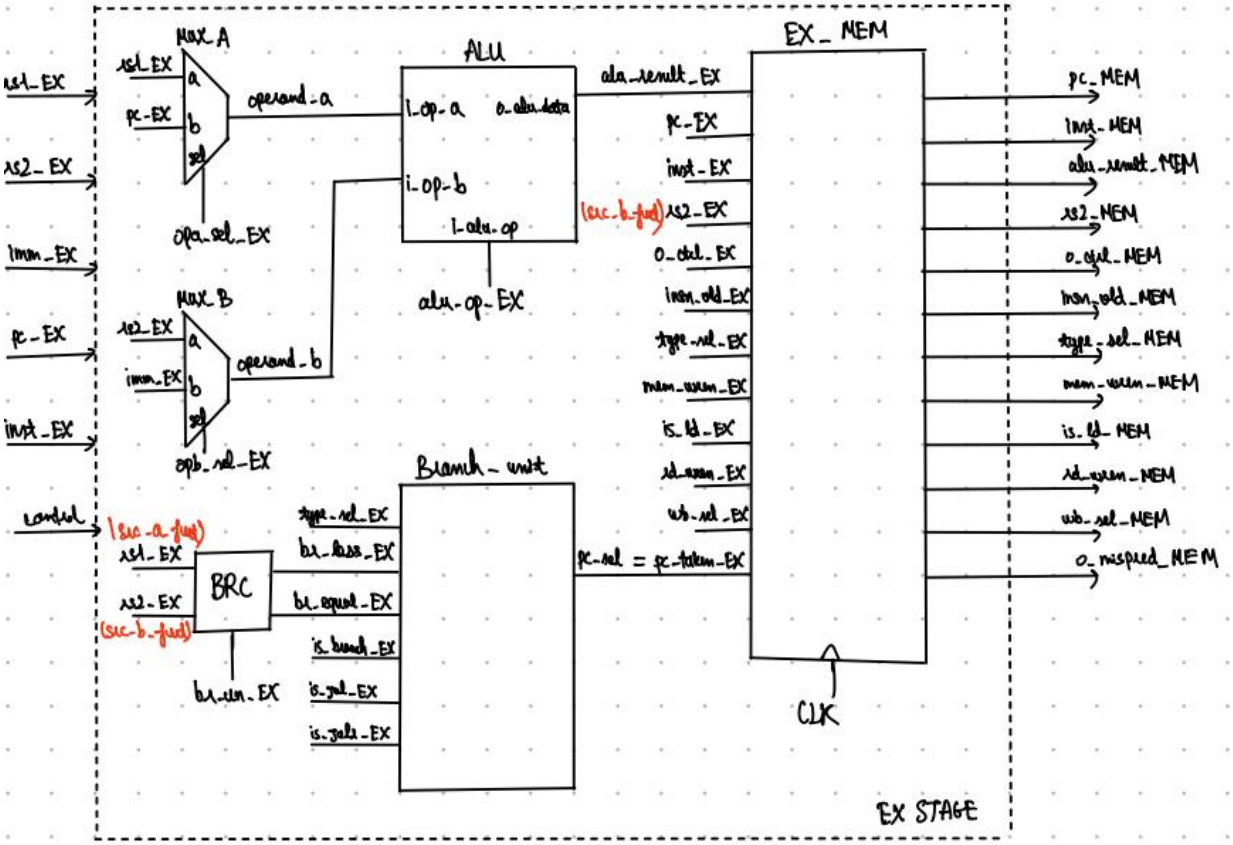
Hình 3: ID Stage

- Trong khối Hazard Unit nhóm sẽ thực hiện so sánh rs1/rs2 với rd ở các pha EX/MEM/WB.

+ Nếu data hazard => stall_PC, stall_ID_IF và flush_ID_EX.

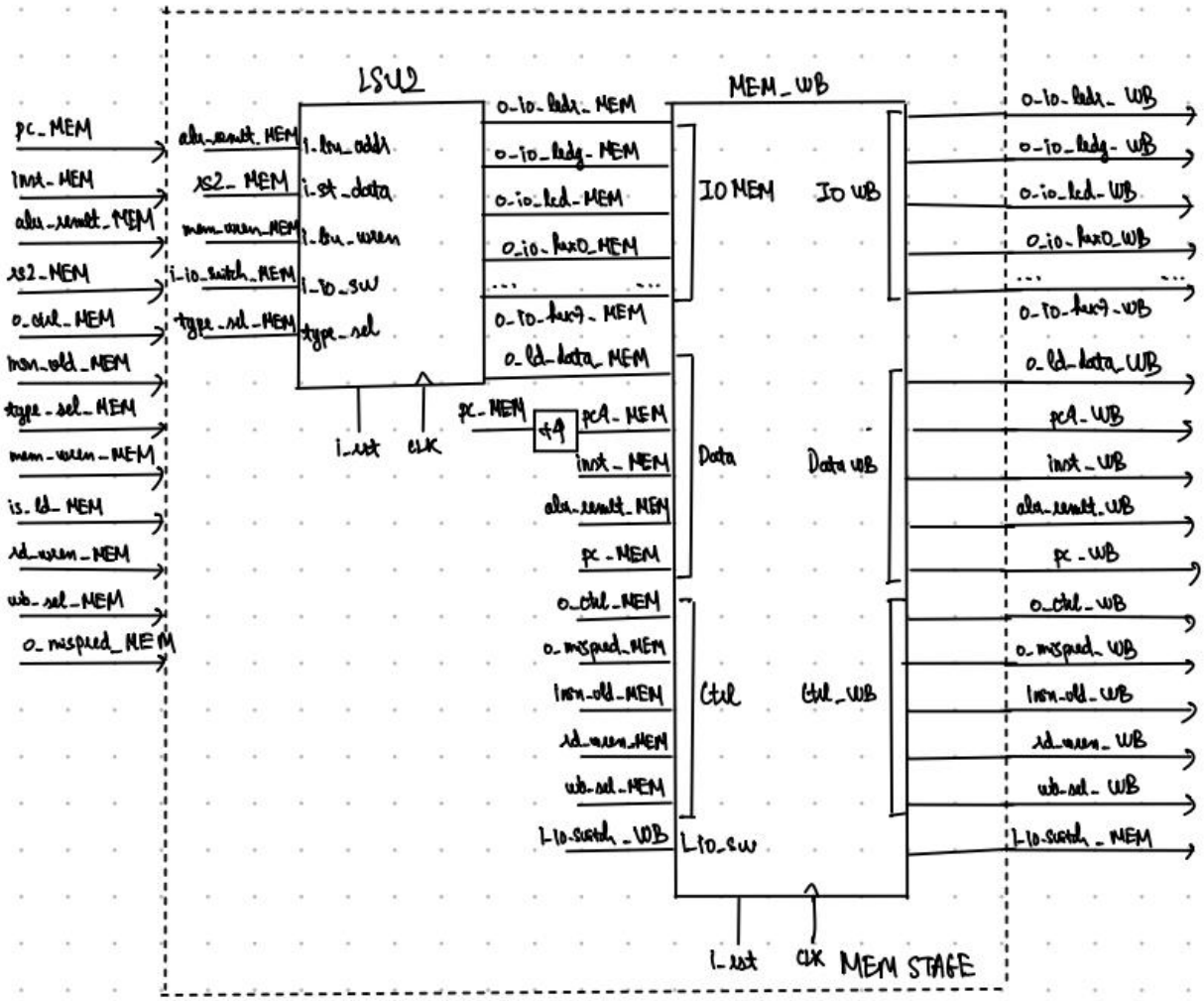
+ Nếu branch_taken => flush_IF_ID, flush_ID_EX.

- <u>EX Stage:</u>



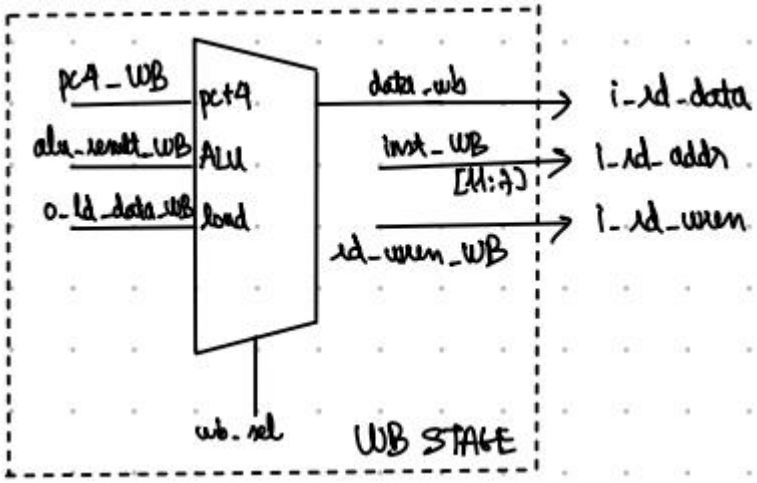
Hình 4: EX Stage

- <u>MEM Stage:</u>

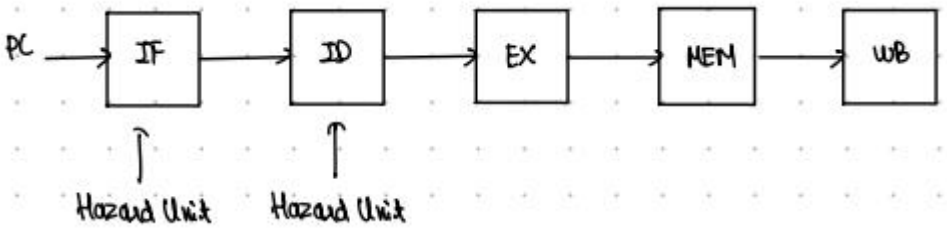


Hình 5: MEM Stage

- <u>WB Stage:</u>

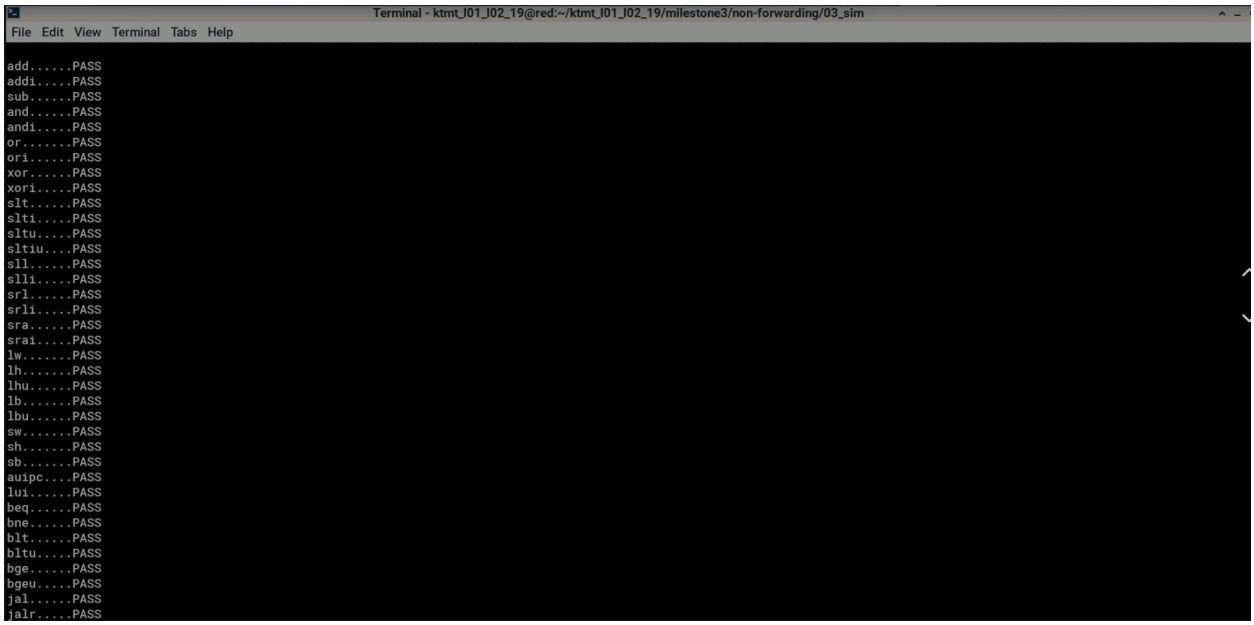


Hình 6: WB Stage



Hình 7: Sơ đồ khối của non - forwarding

❖ Kết quả mô phỏng trên server



Hình 8: Kết quả chạy trên server

- Sau khi chạy mô phỏng, nhóm có kết quả như sau:

```
==================== Result ====================
Total Clock Cycles Executed = 11122
Total Instructions Executed = 4825
Total Branch Instructions   = 1602
Total Branch Mispredictions = 1449

-----------------------------------------------
Instruction Per Cycle (IPC) = 0.43
Branch Misprediction Rate   = 90.45 %

END of ISA tests
```

Hình 9: Kết quả đo trên server

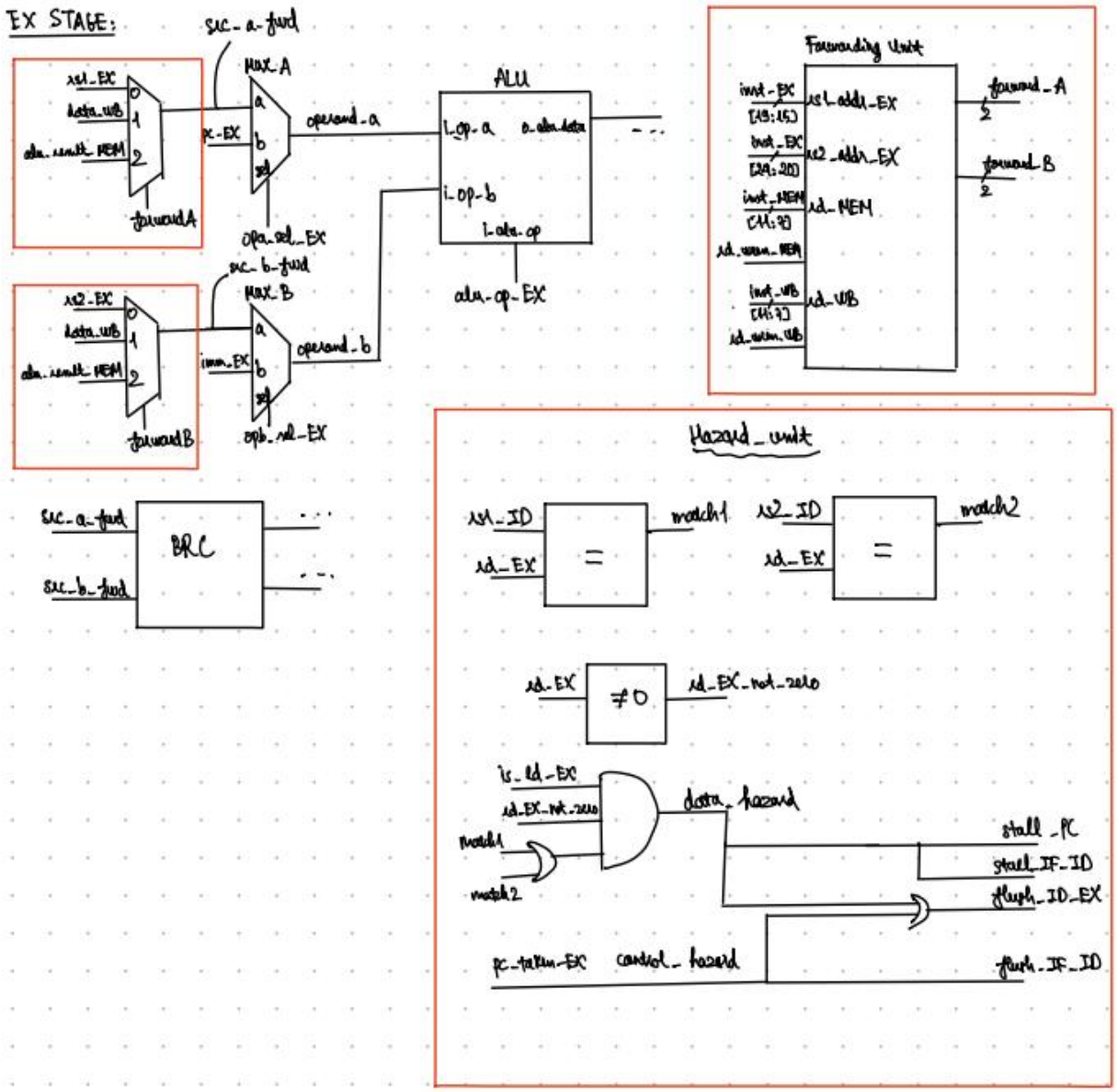
=> Ta thấy IPC khá thấp, chỉ đạt được 0.43 vì ở model này stall và flush khá nhiều làm cho chu kỳ thực hiện của bộ xử lý tăng cao, ngoài ra dự đoán sai lên đến 90.45%. Để cải thiện IPC ta phải giảm bớt stall và flush lại, điều này được thực hiện ở model forwarding dưới đây.

4. Model Forwarding

- Ở model này nhóm thiết kế lại khối regfile khác với ở model trên, khối regfile mới này có khả năng đọc được giá trị i_rd_data bằng cách tạo thêm 2 con mux ở trong băng thanh ghi regfile. 2 con mux này có nhiệm vụ kiểm tra xem các tín hiệu: tín hiệu so sánh địa chỉ của thanh ghi rs1, rs2 với địa chỉ thanh ghi đích; tín hiệu kiểm tra xem đó có phải là thanh ghi x0 hay không và tín hiệu cho phép ghi. Nếu 3 tín hiệu đó đồng thời được cho phép thì đọc dữ liệu mới là i_rd_data mới ghi vào, còn không thì ghi giá trị cũ của rs1 và rs2. Làm như vậy giúp có thể đọc giá trị mới ngay trong chu kỳ đó không phải stall thêm 1 chu kỳ.
- Ngoài ra còn thêm 1 khối forwarding trong model này, khối này có công dụng so sánh địa chỉ thanh ghi nguồn (rs1_EX, rs2_EX) của lệnh đang thực thi với địa chỉ thanh ghi đích với của 2 lệnh trước (rd_MEM, rd_WB). Nếu phát hiện địa chỉ trùng, khác thanh ghi x0 và có tín hiệu cho phép ghi thì sẽ lấy trực tiếp data từ tầng MEM và WB mà không cần phải stall ở chờ thêm chu kỳ tiếp theo. Nhưng đối với các trường hợp Load - Use Hazard thì phải dùng Hazard unit để stall lại như non - forwarding.

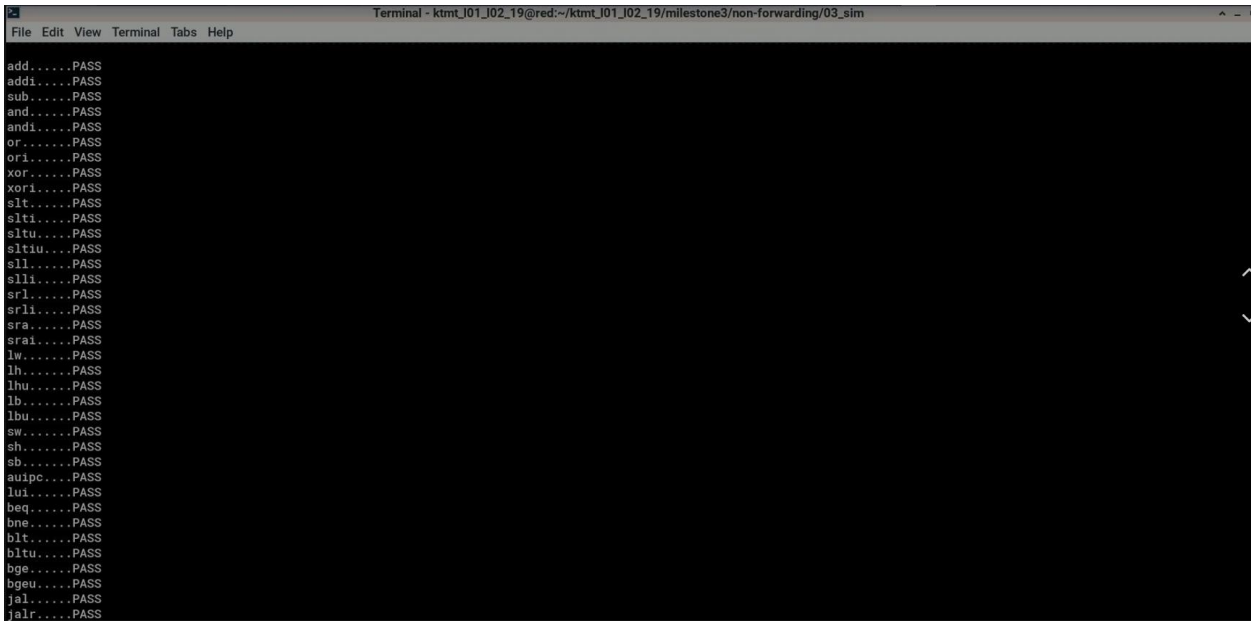
❖ Sơ đồ khối của forwarding

- Sơ đồ khối của fowarding chỉ khác pha EX stage và có thêm khối fowarding



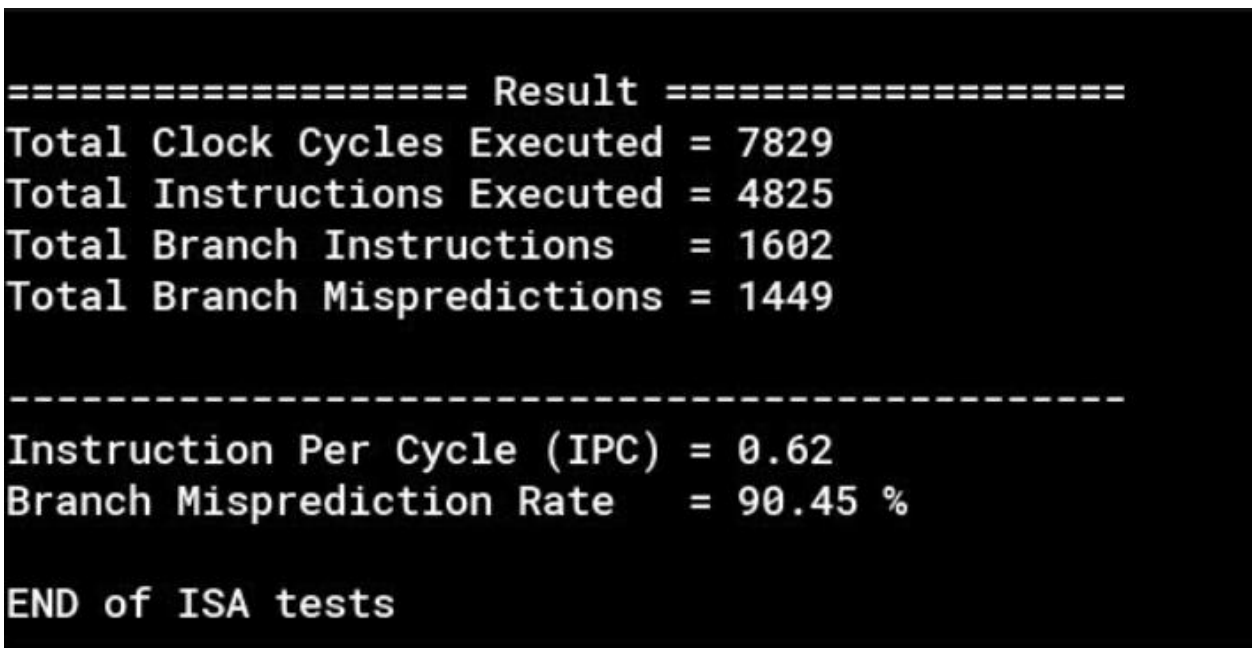
Hình 10: Sơ đồ khối forwarding

❖ Kết quả mô phỏng trên server



Hình 11: Kết quả chạy trên server

- Sau khi chạy mô phỏng, ta có kết quả như sau:



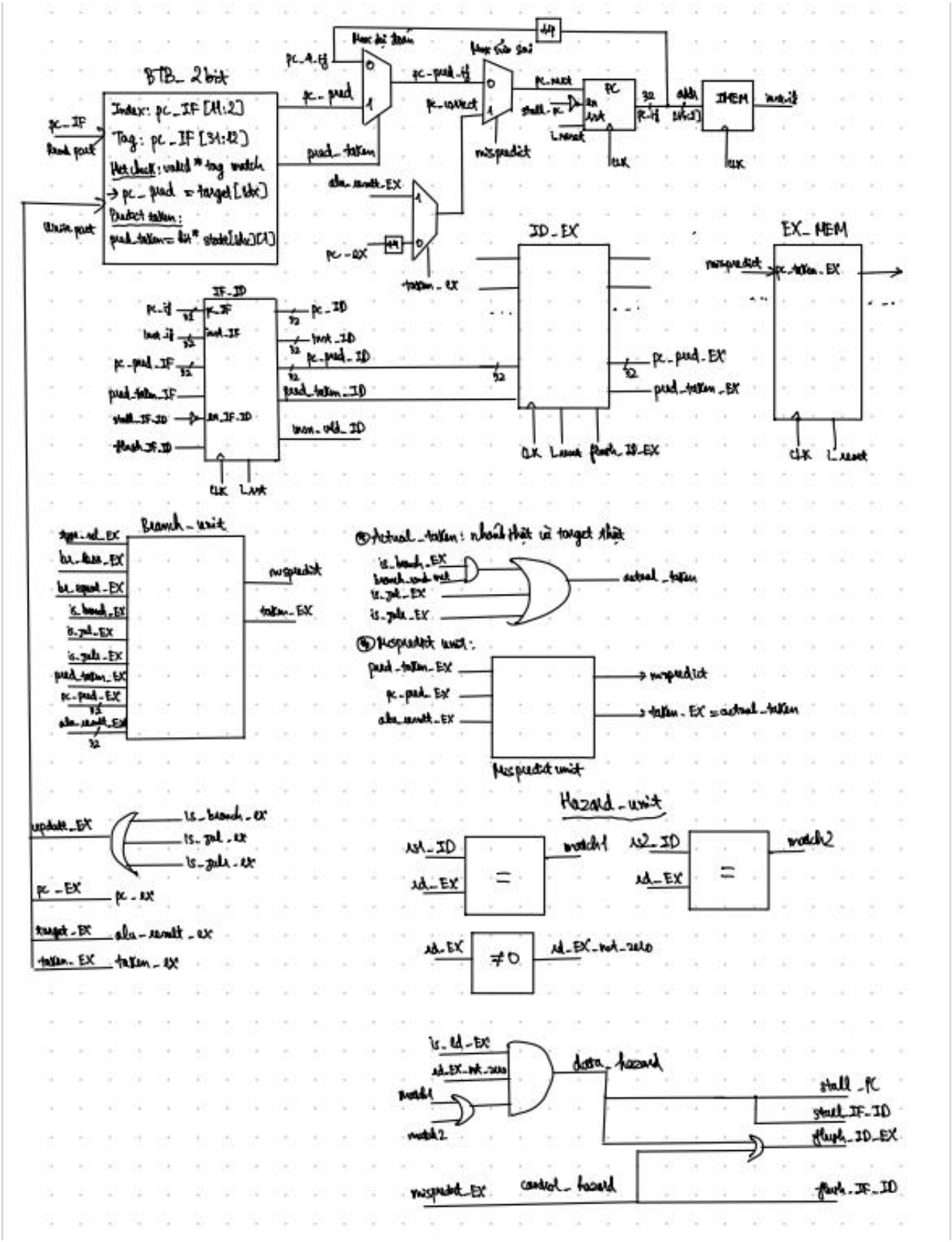
Hình 12: Kết quả đo trên server

=> Ta thấy IPC tăng lên khá nhiều so với mô hình non - forwarding vì các lệnh stall và flush được xử lý một cách tối ưu hơn nên làm cho chu kỳ để chạy giảm đi còn 7829 nhưng tỉ lệ dự đoán sai vẫn còn rất cao, sang mô hình sau nhóm sẽ giảm tỉ lệ này xuống làm tăng thêm IPC.

5. Model Two-bit Dynamic Branch Prediction

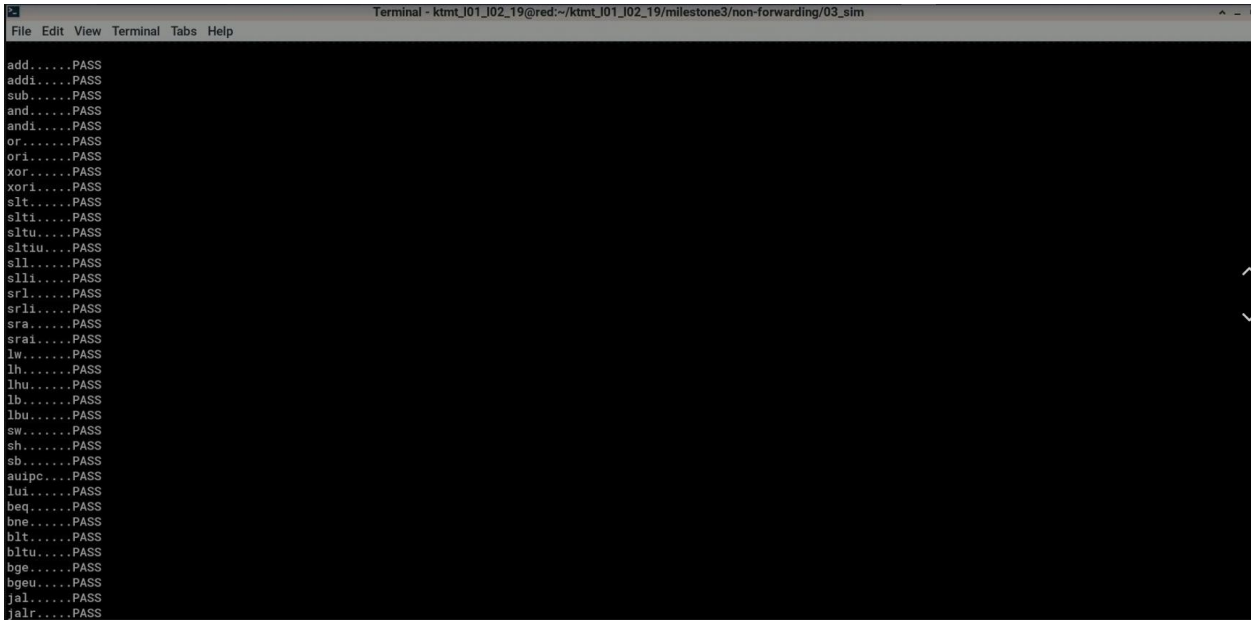
- Ở model này nhóm có thiết kế thêm một khối BTB (Branch Target Buffer) hay có thể coi là bộ nhớ cache, nó là một bộ nhớ dung lượng nhỏ có khả năng dự đoán lệnh nhảy (2-bit). Nó sẽ lưu địa chỉ đích của lệnh nhảy đã từng thực thi vào ô nhớ, khi gặp lại lệnh cũ địa chỉ đích sẽ có ngay lập tức mà không cần tính toán lại. Two-bit ở đây dùng để biểu diễn trạng thái nhảy, nếu 2bit là 11 thì chắc chắn sẽ nhảy nhưng nếu dự đoán đó sai thì sẽ giảm đi một còn lại 10 (vẫn đoán là nhảy nhưng sẽ không còn chắc chắn) và ngược lại thì 00 chắc chắn sẽ không nhảy nhưng khi dự đoán sai sẽ tăng lên một sẽ là 01 (chưa chắc là không nhảy). Khối này giúp dự đoán từ sớm ngay từ pha IF chứ không cần phải chờ đến pha EX làm mất 2 chu kỳ.

❖ **Sơ đồ khối của thiết kế**



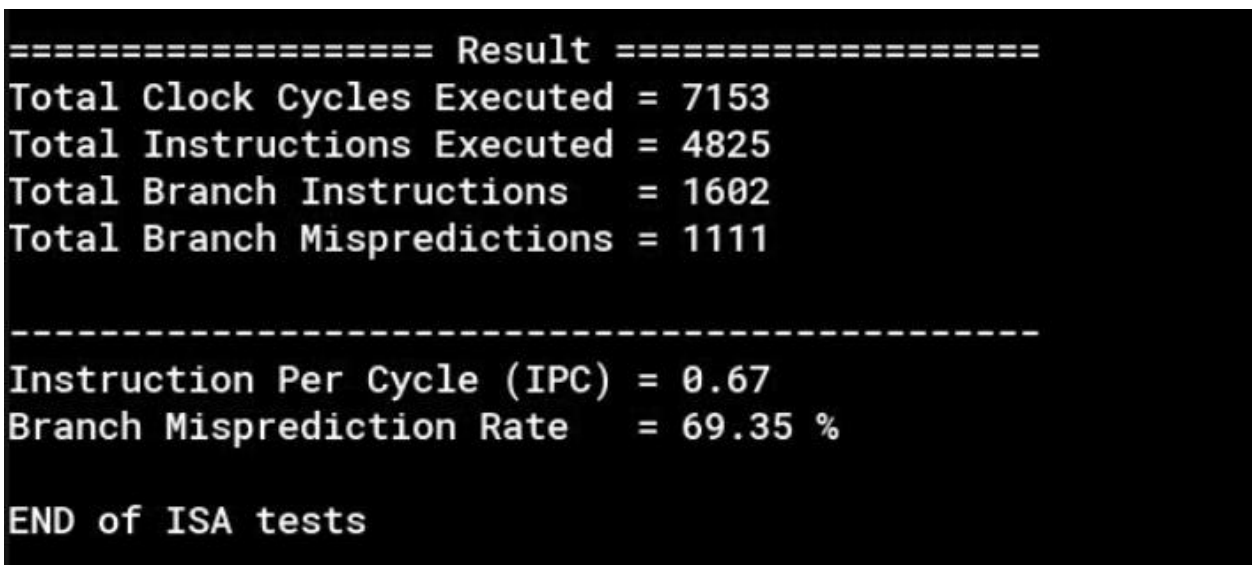
Hình 13: Sơ đồ khối Two-bit Dynamic Branch Prediction

❖ Kết quả mô phỏng trên server



Hình 14: Kết quả chạy trên server

- Sau khi chạy mô phỏng, ta có kết quả như sau:



Hình 15: Kết quả chạy trên server

=> Từ kết quả trên ta thấy IPC tăng so với mô hình trước vì tỉ lệ dự đoán sai của mô hình đã giảm còn 69.35% so với 90.45% của 2 mô hình trên nhưng IPC chưa đạt được giá trị lý tưởng là 1. Nhóm sẽ cố gắng thực hiện thêm các mô hình khác để cải thiện thêm IPC.

6. Kết quả so sánh với các mô hình đã làm

❖ **Bảng so sánh hiệu năng của bộ xử lý pipelined**

Model (ISA test)	Non - forwarding	Forwarding	Two - bit
IPC	0.43	0.62	0.67
Mispred Rate (%)	90.45	90.45	69.35

=> Từ bảng trên ta thấy được model Two-bit có IPC đang cao nhất trong 3 model và tỉ lệ dự đoán sai cũng thấp nhất. Ta thấy Non - forwarding có kiến trúc đơn giản nhất nhưng hiệu năng lại thấp nhất còn Two - bit tuy phức tạp hơn nhưng hiệu năng lại cao nhất vì vậy Two - bit phù hợp với các ứng dụng yêu cầu tốc độ xử lý cao.

III. Đánh giá

Nhóm đã thiết kế được 3 mô hình giúp tối ưu hơn so với single cycle processor ở milestone 2, giúp bộ xử lý có thể xử lý được nhiều câu lệnh cùng lúc làm tăng hiệu năng của bộ xử lý. Milestone này nhóm đã áp dụng các kĩ thuật để giúp bộ xử lý có thể đạt được hiệu năng cao hơn như dùng forwarding hay là khối BTB. Mọi người trong nhóm đã phối hợp rất tốt với nhau để thực hiện được milestone này và cũng như hiểu được các mô hình mà mình đã làm ra.